GRUPO 6 · REQUERIMIENTOS FUNCIONALES

Notificaciones y Alertas

Módulo: Sistema de notificaciones (push / email / SMS)

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 6
Versión	1.0
Fecha	22 de agosto de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST) y aiquaa-sandbox-web (front)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre la creación de notificaciones dirigidas a un titular por alguno de los tres canales disponibles, su consulta con filtros por titular y por estado de lectura, y el marcado como leída. El sandbox registra la notificación y la da por enviada: no existe integración con ningún proveedor real de correo, SMS o push.

Fuera de alcance: Envío efectivo del mensaje, reintentos ante fallo, plantillas, programación diferida, preferencias de contacto del titular y baja de suscripción.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con su propia API key y automatiza escenarios BDD sobre el módulo de su grupo.
Usuario de negocio (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es dueño de cuentas, facturas, tarjetas, órdenes, reservas, notificaciones y roles.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz que consume la API a través del proxy `/api/proxy/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v1/notificaciones	RF-G6-01	Lista notificaciones activas con filtros por titular y por leídas / no leídas.
POST /api/v1/notificaciones	RF-G6-02	Crea una notificación y la deja como enviada.
GET /api/v1/notificaciones/{id}	RF-G6-04	Devuelve el detalle de una notificación.
PUT /api/v1/notificaciones/{id}	RF-G6-05	Reemplaza canal, asunto y mensaje de una notificación.
DELETE /api/v1/notificaciones/{id}	RF-G6-06	Da de baja (soft-delete) una notificación.
PATCH /api/v1/notificaciones/{id}/leer	RF-G6-03	Marca una notificación como leída.

2. Precondiciones y acceso

- Toda request a la API debe enviar el header `x-api-key` con una clave existente y con `active = true` en la tabla `public.api_keys`; sin header o con clave inválida/inactiva la respuesta es `401 UNAUTHORIZED` con el mensaje genérico "Invalid or inactive API key.".
- Cada API key admite un máximo de **200 requests por minuto** (ventana deslizante). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.

- Los datos de práctica viven en el schema aislado `qa_training` y se cargan con `scripts/seed-data.sql`; la data es determinística, por lo que un escenario puede apoyarse en los registros sembrados.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` de la API hace baja lógica (`activo = false`), devolviendo `204 No Content` sin cuerpo.
- Casi todas las tablas de negocio (`sesiones`, `transferencias`, `facturas`, `tarjetas`, `notificaciones`, `ordenes`, `reservas`, `movimientos`, `roles`, además de `usuarios` y `cuentas` que ya la tenían) tienen columna `activo` / `activa`; todo `GET`, `PUT` y `DELETE` de esa tabla filtra u opera solo sobre filas con `activo = true` — una fila dada de baja deja de ser visible y de poder reemplazarse, pero no se borra.
- En la aplicación web el acceso tiene dos capas: capa 1 la API key (pantalla `/login`, se valida contra `GET /api/v1/roles` antes de guardarse) y capa 2 el usuario de negocio (pantalla `/auth/login`). Con `NEXT_PUBLIC_DEMO_MODE=true` la capa 1 se omite y el proxy inyecta una key demo del servidor.
- Las rutas `/usuarios/*` de la web son la excepción al guard de capa 2: se pueden usar solo con la API key, porque son el punto de partida para obtener un `usuarioId`.
- La creación requiere un titular existente; el `usuarioId` se obtiene del alta de cliente (RF-G4-01) o de los datos sembrados.
- Los datos sembrados incluyen notificaciones leídas y no leídas en los tres canales, suficientes para probar los filtros.

3. Modelo de datos del módulo

notificaciones

Mensajes dirigidos a un titular. Es la única tabla que toca el módulo.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`canal`	`text`, uno de `push` / `email` / `sms`; por defecto `email`
`asunto`	`text`, obligatorio
`mensaje`	`text`, obligatorio
`leido`	`boolean`, obligatorio, por defecto `false`
`estado`	`text`, uno de `enviada` / `fallida` / `pendiente`; por defecto `enviada`
`created_at`	`timestamptz`, por defecto `now()`
`activo`	`boolean`, obligatorio, por defecto `true`; lo usan `GET/PUT/DELETE` para filtrar/operar

> Conviene no confundir las dos dimensiones de estado: `estado` describe el envío (siempre `enviada` para lo que crea la API) y `leido` describe la lectura por parte del destinatario. Solo la segunda cambia con el uso.

4. Requerimientos funcionales

RF-G6-01 — Listar notificaciones

`GET /api/v1/notificaciones`

Devuelve las notificaciones del sandbox, con filtros combinables por titular y por condición de lectura. Es el requerimiento que alimenta la bandeja del usuario.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.
- `leido` (opcional, por query): exactamente el texto `true` o el texto `false`. No se aceptan otras formas de expresar el valor booleano (`1`, `0`, `sí`, `no`).

Reglas de negocio

- Devuelve únicamente notificaciones con `activo = true`; una dada de baja con RF-G6-06 deja de aparecer.
- Los dos filtros son opcionales y se combinan con Y lógico.
- El filtro de lectura acepta únicamente los textos `true` y `false`; cualquier otro valor es rechazado como error de validación, en lugar de interpretarse como verdadero.
- El resultado se ordena por identificador ascendente y está limitado a 100 registros.
- Sin filtros, devuelve las notificaciones de todos los titulares: la lista no está acotada al usuario en sesión.

Respuesta esperada

- `200 OK` con `data` como arreglo de notificaciones, cada una con `id`, `usuario_id`, `canal`, `asunto`, `mensaje`, `leido`, `estado` y `created_at`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`leido` con un valor distinto de `true` o `false`, o `usuarioId` no numérico.
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/notificaciones/route.ts`

RF-G6-02 — Crear una notificación

`POST /api/v1/notificaciones`

Registra una notificación dirigida a un titular por un canal determinado, con su asunto y su mensaje. Queda marcada como enviada y sin leer.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo del destinatario.
- `canal` (obligatorio): uno de `push`, `email` o `sms`.
- `asunto` (obligatorio): texto no vacío.
- `mensaje` (obligatorio): texto no vacío.

Reglas de negocio

- La notificación se crea siempre con `estado = 'enviada'` y `leido = false`; ninguno de los dos puede enviarse desde el cliente.
- **No hay envío real:** el estado `enviada` describe el registro, no una entrega confirmada por un proveedor.
- Los estados `fallida` y `pendiente` existen en el modelo pero la API nunca los produce.
- El destinatario debe existir: un `usuarioId` inexistente falla como error de ejecución por violación de clave foránea.
- No se valida que el titular tenga un dato de contacto para el canal elegido (por ejemplo, un teléfono para `sms`), porque el modelo no los almacena.
- El asunto y el mensaje deben tener al menos un carácter, pero no tienen tope de longitud.

Respuesta esperada

- `201 Created` con `data` conteniendo la notificación creada.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, `canal` no está permitido, asunto/mensaje están vacíos, o el destinatario indicado no existe.

Fuente: `app/api/v1/notificaciones/route.ts`

RF-G6-04 — Consultar una notificación

`GET /api/v1/notificaciones/{id}`

Devuelve el detalle de una notificación puntual.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve la notificación cuyo identificador coincide exactamente, siempre que siga `activo = true`.
- Si no existe o ya está dada de baja, responde 404 con el mensaje "Notificación no encontrada.".

Respuesta esperada

- `200 OK` con `data` conteniendo la notificación completa.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una notificación vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/notificaciones/[id]/route.ts`

RF-G6-05 — Reemplazar una notificación

`PUT /api/v1/notificaciones/{id}`

Reemplaza el canal, el asunto y el mensaje de una notificación existente. La condición de lectura y el estado de envío quedan fuera de este reemplazo.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `canal` (obligatorio): uno de `push`, `email`, `sms`.
- `asunto` (obligatorio): texto no vacío.
- `mensaje` (obligatorio): texto no vacío.

Reglas de negocio

- Solo puede reemplazar una notificación con `activo = true`; una dada de baja responde 404.
- `leido` y `estado` no forman parte del cuerpo: marcar como leída sigue siendo exclusivo de RF-G6-03.
- Reemplazar una notificación ya leída no la vuelve a marcar como no leída.

Respuesta esperada

- `200 OK` con `data` conteniendo la notificación ya actualizada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una notificación vigente con ese identificador.
`VALIDATION_ERROR`	400	Falta un campo obligatorio, o `canal` tiene un valor fuera de la lista permitida.

Fuente: `app/api/v1/notificaciones/[id]/route.ts`

RF-G6-06 — Dar de baja una notificación

`DELETE /api/v1/notificaciones/{id}`

Da de baja lógica una notificación, quitándola de los listados y consultas por id.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Marca `activo = false`; la fila permanece en la base, nunca se borra físicamente.
- Una notificación ya leída también puede darse de baja.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una notificación vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/notificaciones/[id]/route.ts`

RF-G6-03 — Marcar una notificación como leída `PATCH /api/v1/notificaciones/{id}/leer`

Registra que el destinatario leyó la notificación, para que deje de figurar entre las pendientes de lectura.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo de la notificación.

Reglas de negocio

- La notificación pasa a `leido = true` sin verificar su valor anterior.
- La operación es idempotente: marcar como leída una notificación ya leída devuelve el mismo resultado, sin error.
- **No se verifica quién marca la lectura:** cualquier portador de una API key válida puede marcar como leída una notificación de cualquier titular.
- No existe la operación inversa: una vez leída, la notificación no puede volver a marcarse como no leída por API.
- No se registra la fecha de lectura, solo el hecho de que fue leída.
- **No filtra por `activo`:** a diferencia de `GET` / `PUT` / `DELETE`, este endpoint no se tocó al agregar el CRUD, así que puede marcar como leída una notificación ya dada de baja con RF-G6-06, aunque esa notificación ya no aparezca en ningún listado.
- Si la notificación no existe (ID inexistente), responde 404 y no modifica nada.

Respuesta esperada

- `200 OK` con `data` conteniendo la notificación ya marcada como leída.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una notificación con ese identificador (mensaje: "Notificación no encontrada.").
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/notificaciones/[id]/leer/route.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo).
- **Coerción de tipos:** los identificadores y montos que llegan por query o por path se convierten de texto a número automáticamente; los campos numéricos de un cuerpo JSON deben enviarse como número, no como texto, salvo que el requerimiento indique lo contrario.

- **Cuerpo JSON malformado:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body.".
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`, ej. un email o número de factura repetido) se traduce a `409 CONFLICT`; una violación de clave foránea o de restricción `CHECK` (`23503` / `23514`) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos cae en `400 EXECUTION_ERROR`.
- `DELETE` **es siempre baja lógica:** actualiza `activo = false` (o `activa` en `cuentas`) y responde `204 No Content` **sin cuerpo** — no `200` con la fila afectada.
- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera Next.js automáticamente, ninguna ruta lo arma a mano, y su cuerpo no sigue la envoltura `{ error }` del resto de la API.
- **Auditoría:** toda request, exitosa o fallida, deja un registro en `public.sql_audit_log` con la clave usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Idempotencia de las transiciones de estado:** los endpoints que fijan un estado (`bloquear`, `activar`, `confirmar`, `cancelar`, `leer`) escriben el valor destino sin verificar el estado previo, por lo que repetir la llamada devuelve el mismo resultado.
- **Las transiciones de estado anteriores al CRUD no miran** `activo`**:** `PATCH .../bloquear`, `.../activar`, `.../leer`, `.../confirmar`, `.../cancelar`, `.../kyc` y `POST .../pagar` no filtran por `activo = true` (ninguno de esos archivos se tocó al agregar el CRUD). Un recurso dado de baja con el nuevo `DELETE` deja de aparecer en `GET`, pero **sigue pudiendo bloquearse, confirmarse, leerse o pagarse** a través de su acción dedicada — una inconsistencia real del sandbox, no un comportamiento a asumir como intencional.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso; también un POST de upsert que reactivó una fila existente en vez de crearla.
`201 Created`	POST que crea una fila nueva.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Falta el header `x-api-key`, o la clave no existe o está inactiva.
`RATE_LIMITED`	429	Se superaron las 200 requests por minuto de esa API key.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe, ya está dado de baja (`activo=false`), o no está en un estado que admita la operación.
`CONFLICT`	409	Violación de restricción `UNIQUE` en Postgres (ej. email, número de factura o nombre de rol duplicado).
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú de navegación muestra únicamente los módulos del grupo del alumno cuando su email figura en el roster del curso; si no figura, muestra los diez módulos. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local. **La interfaz web todavía no tiene pantallas para `PUT` / `DELETE` ni para los recursos nuevos (`sesiones`, `movimientos`)**: solo cubre los flujos de creación y consulta que ya existían; esos requerimientos se prueban llamando la API directamente (Postman, `curl`, o el cliente HTTP del framework de automatización), no navegando la web.

Pantalla	Qué permite hacer
`/notificaciones` — Bandeja	Tabla con canal, asunto, mensaje y condición de lectura, con filtros por titular y por leídas / no leídas, y un botón por fila para marcar como leída. Ejecuta RF-G6-01 y RF-G6-03.
`/notificaciones/new` — Nueva notificación	Formulario con destinatario, canal, asunto y mensaje. Ejecuta RF-G6-02 y, al crearse, vuelve a la bandeja.

- El filtro de lectura de la pantalla ofrece tres opciones (todas, leídas, no leídas) y envía exactamente los textos `true` y `false` que acepta la API.
- Marcar una notificación como leída refresca la bandeja, por lo que la fila cambia de estado sin recargar la página.
- No hay pantalla de detalle, edición ni baja individual: `GET`, `PUT` y `DELETE /notificaciones/{id}` (RF-G6-04/05/06) solo se pueden ejercitar llamando la API directamente.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G6-01 — Listar notificaciones

- Al listar con `leido=false`, todos los elementos devueltos tienen `leido` en falso.
- Al listar con `leido=true`, todos los elementos devueltos tienen `leido` en verdadero.
- Al combinar `usuarioId` y `leido`, todos los elementos cumplen ambas condiciones.
- Al listar con `leido=1`, la respuesta es 400 `VALIDATION_ERROR`: el filtro no acepta otras formas del valor booleano.
- Al filtrar por un titular sin notificaciones, la respuesta es 200 con `data` vacío.

RF-G6-02 — Crear una notificación

- Al crear una notificación con datos válidos, la respuesta es 201, `data.estado` es `enviada` y `data.leido` es falso.
- La notificación recién creada aparece en el listado del titular filtrando por `leido=false`.
- Al crear con `canal=whatsapp`, la respuesta es 400 `VALIDATION_ERROR`.
- Al crear con `asunto` vacío, la respuesta es 400 `VALIDATION_ERROR`.
- Al crear para un `usuarioId` inexistente, la respuesta es 400 `VALIDATION_ERROR` y no se registra ninguna notificación.
- Al crear dos notificaciones idénticas seguidas, ambas son aceptadas y se registran por separado: no hay control de duplicados.

RF-G6-04 — Consultar una notificación

- Dada una notificación existente, al consultarla por su `id` la respuesta es 200 y los datos coinciden con los de la creación.
- Al consultar una notificación inexistente o dada de baja, la respuesta es 404 `NOT_FOUND`.

RF-G6-05 — Reemplazar una notificación

- Dada una notificación existente, al reemplazar su `mensaje` la respuesta es 200 y el cambio se refleja al consultarla.
- El campo `leido` de una notificación ya leída no cambia al reemplazarla.
- Al reemplazar una notificación inexistente o dada de baja, la respuesta es 404 `NOT_FOUND`.

RF-G6-06 — Dar de baja una notificación

- Dada una notificación vigente, al darla de baja la respuesta es 204 sin cuerpo.
- Tras la baja, `GET /notificaciones/{id}` sobre esa notificación responde 404, y deja de aparecer en RF-G6-01.

RF-G6-03 — Marcar una notificación como leída

- Dada una notificación no leída, al marcarla la respuesta es 200 y `data.leido` es verdadero.
- Tras marcarla, deja de aparecer en el listado filtrado por `leido=false` y aparece en el de `leido=true`.

- Al marcar dos veces la misma notificación, ambas respuestas son 200 y el resultado final es el mismo.
- Al marcar una notificación inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Notificación no encontrada.".
- El campo `estado` de la notificación no cambia al marcarla como leída: sigue siendo `enviada`.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Ley 1334/98 de Defensa del Consumidor (SEDECO), deber de información	El proveedor debe informar al consumidor sobre hechos relevantes de su servicio de manera oportuna y comprobable.	Parcial: la notificación queda registrada y es consultable, pero no hay entrega real ni constancia de recepción.
Ley 4868/2013 de Comercio Electrónico, comunicaciones al usuario	Las comunicaciones deben identificar al remitente y permitir al destinatario dejar de recibirlas.	No implementado: no hay remitente en el modelo ni mecanismo de baja de suscripción.
Marco del BCP sobre alertas de operaciones en canales electrónicos	Las operaciones sensibles deben disparar una alerta al titular por un canal previamente registrado.	No implementado: ninguna operación del sandbox (transferencia, pago, emisión de tarjeta) genera una notificación automática.
Ley 6534/2020 de protección de datos personales crediticios	El contenido dirigido a un titular solo debe ser accesible para ese titular.	No implementado: el listado sin filtro devuelve notificaciones de todos los titulares y cualquiera puede marcarlas como leídas.
Preferencias de contacto del titular	El canal utilizado debe corresponder a un dato de contacto validado y consentido por el titular.	No implementado: el canal se elige en la request y el modelo no guarda teléfono ni preferencias.

Brechas conocidas del sandbox

- No hay envío real por ningún canal: `estado = 'enviada'` describe el registro, no una entrega.
- Los estados `fallida` y `pendiente` nunca se producen, por lo que no hay escenarios de reintento.
- Cualquier portador de una API key puede leer y marcar notificaciones de cualquier titular: no hay aislamiento por destinatario.
- No se puede desmarcar una notificación leída ni se registra la fecha de lectura.
- Ninguna operación de negocio del sandbox genera notificaciones automáticas.
- El listado está limitado a 100 registros sin paginación.
- `PATCH .../leer` no filtra por `activo`: puede marcar como leída una notificación ya dada de baja con `DELETE`.